

Claudia Melati Krid - 18224032

Teknik Watermarking

Teknik watermark yang dipakai adalah **DCT** (Discrete Cosine Transform) dengan cara:

1. Membagi gambar menjadi blok berukuran 8*8 pixel.
2. Setiap blok akan diubah ke domain frekuensi menggunakan rumus DCT.
3. Watermark disisipkan ke koefisien frekuensi menengah. Koefisien frekuensi menengah dipilih karena cukup *invisible* namun masih tahan kompresi.
4. Untuk tugas kali ini, saya menggunakan watermark berupa **citra acak (*random PN sequence*)**
5. Setelah disisipkan watermark, gambar didekompresi kembali ke domain spasial menggunakan *Inverse DCT*

Penjelasan Prosedur

1. Import

```
1 import argparse
2 import io
3 import os
4
5 import cv2
6 import matplotlib.pyplot as plt
7 import numpy as np
8 from PIL import Image
```

Pada bagian ini, dijalankan import untuk menambahkan library seperti:

- Argparse: membaca argument terminal
- io: buffer memory
- os: operasi file / folder
- cv2: untuk memproses gambar dan DCT
- matplotlib: untuk memplotting grafik outputnya nanti
- numpy: untuk men-generate matrix
- PIL: untuk menyimpan JPEG dengan Quality Factor (QF) tertentu sebagai output

2. Watermark Generation

```
def generate_random_watermark(size=(32, 32), seed=42):
    """Pseudo-random binary watermark (seed = secret key)."""
    rng = np.random.default_rng(seed)
    return rng.integers(0, 2, size=size).astype(np.float32)
```

Potongan kode dari fungsi generate_random_watermark pada bagian Watermark Generation

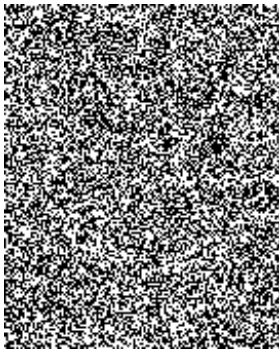
```

H_img, W_img = image.shape[:2]
wm_size = (H_img // 8, W_img // 8)
if watermark_type == "binary":
    watermark = generate_binary_watermark(wm_size)
    print("[INFO] Using binary (checkerboard) watermark")
else:
    watermark = generate_random_watermark(wm_size, seed=seed)
    print(f"[INFO] Using random watermark (seed={seed})")

```

Potongan kode dari fungsi evaluate pada bagian Main Evaluation Pipeline

Pada random watermark generation, watermark akan dibuat secara pseudo-random yang dapat dilihat pada bagian `rng = np.random.default_rng(seed)`. Ukuran watermark yang dihasilkan disesuaikan dengan ukuran gambar melalui kalkulasi pada `wm_size = (H // 8, W // 8)`, satu bit per blok DCT 8*8. Watermark **citra acak** untuk foto yang saya gunakan (`img_claudia.jpg`) yang sudah di-generate dan akan disisipkan nantinya dapat dilihat pada gambar berikut:



Watermark yang di-generate dan akan digunakan

3. DCT Embedding

```

MID_FREQ_POSITIONS = [(3, 2), (2, 3), (4, 1), (1, 4), (3, 3), (4, 2), (2, 4)]

```

Kemudian, bagian ini adalah untuk menentukan posisi frekuensi gambar yang akan diberi watermark yaitu pada area frekuensi menengah karena koefisien frekuensi menengah dipilih karena cukup *invisible* namun masih tahan kompresi.

```

def embed_dct(image_bgr, watermark, alpha=20):
    img = image_bgr.copy()
    ycrb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb).astype(np.float32)
    Y = ycrb[:, :, 0]

    H, W = Y.shape
    wm_bits = watermark.flatten()
    num_bits = len(wm_bits)

```

Gambar kemudian diubah menjadi YCrCb karena watermark akan disisipkan pada channel Y (luminance) karena *brightness* (Y) lebih sensitif dan JPEG bekerja dominan di Y sehingga watermark akan lebih signifikan.



Gambar Asli



Gambar yang Diubah menjadi YCrCb

```
bit_idx = 0
blocks_used = 0

for row in range(0, H - 7, 8):
    for col in range(0, W - 7, 8):
        if bit_idx >= num_bits:
            break
        block = Y[row:row+8, col:col+8]
        dct_block = cv2.dct(block)

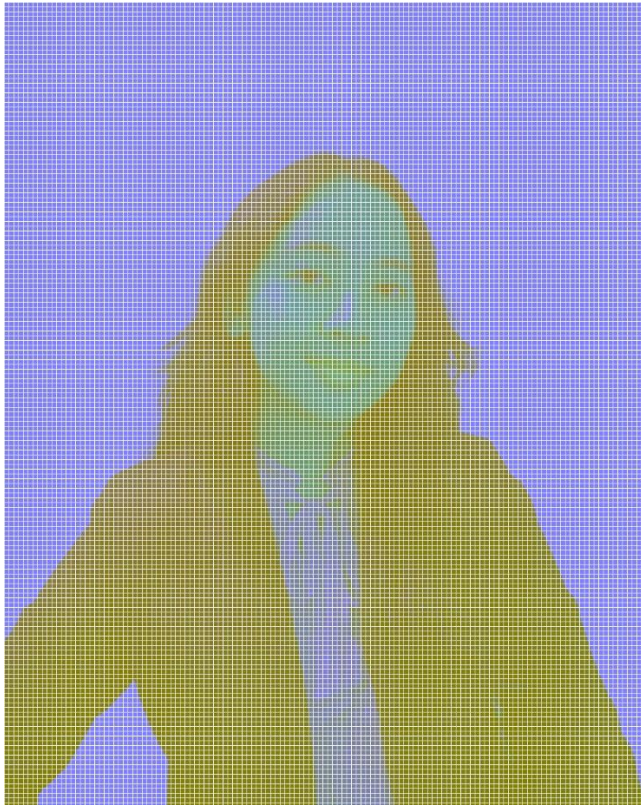
        pos = MID_FREQ_POSITIONS[bit_idx % len(MID_FREQ_POSITIONS)]
        bit = wm_bits[bit_idx]

        if bit == 1:
            dct_block[pos] += alpha
        else:
            dct_block[pos] -= alpha

        Y[row:row+8, col:col+8] = cv2.idct(dct_block)
        bit_idx += 1
        blocks_used += 1

ycrcb[:, :, 0] = np.clip(Y, 0, 255)
watermarked_bgr = cv2.cvtColor(ycrcb.astype(np.uint8), cv2.COLOR_YCrCb2BGR)
return watermarked_bgr, wm_bits, blocks_used
```

Setelah dipisah menjadi YCrCb, akan diloop per blok 8*8 untuk discan kemudian akan ditransformasi ke domain frekuensi (`dct_block = cv2.dct(block)`).



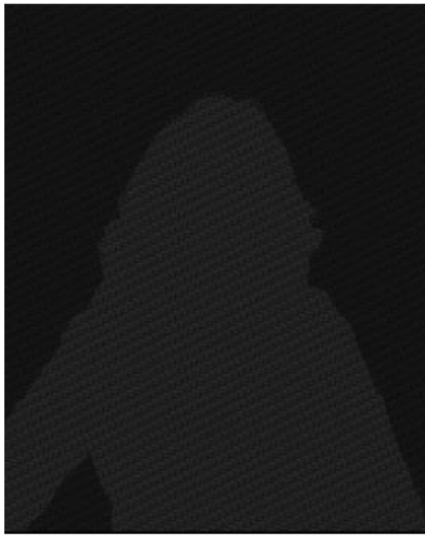
Gambar YCrCb yang Dibagi ke dalam Blok 8*8 Pixel

Lalu, koefisien DCT akan dimodifikasi dengan menambahkan alpha. Terakhir, pada baris `cv2.idct(dct_block)`, domain frekuensi dikembalikan ke domain gambar dan akan menghasilkan output berupa gambar yang sudah diberi watermark.



Gambar yang Sudah Diberi Watermark

Berikut gambaran perbedaan pixel antara gambar yang sudah diberi watermark dengan gambar original:



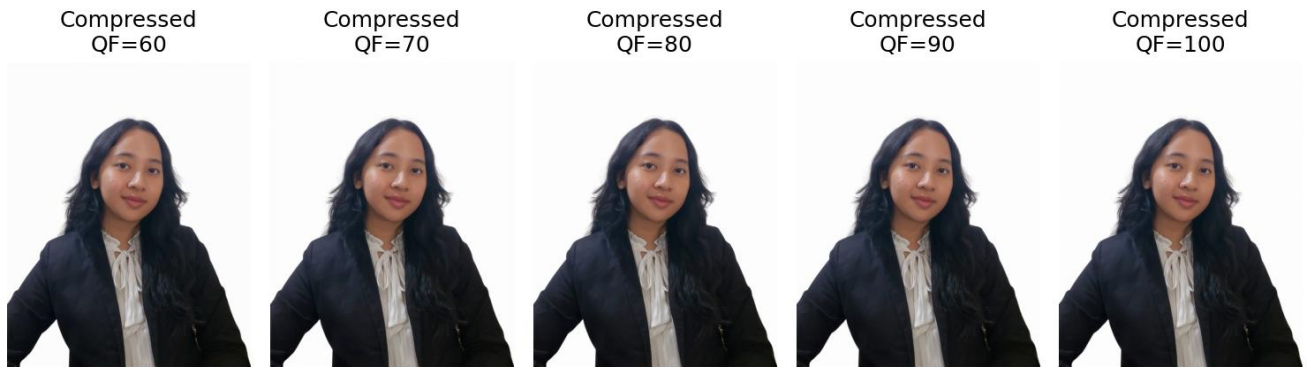
Difference map → pada gambar ini, selisih diperkuat x15 supaya selisih piksel terlihat lebih jelas. Untuk yang berwarna putih (lebih terang) berarti perbedaan piksel pada area tersebut lebih besar. Sedangkan untuk yang berwarna hitam, artinya perubahan kecil.

4. JPEG Compression

```
def jpeg_compress(image_bgr, quality):  
    img_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)  
    pil_img = Image.fromarray(img_rgb)  
    buffer = io.BytesIO()  
    pil_img.save(buffer, format="JPEG", quality=quality)  
    buffer.seek(0)  
    decompressed = np.array(Image.open(buffer))  
    return cv2.cvtColor(decompressed, cv2.COLOR_RGB2BGR)
```

Fungsi jpeg_compress berperan untuk mengompres JPEG dengan Quality Factor (QF) tertentu, yaitu pada QF=10 hingga QF=100 dengan interval 5. Jika QF kecil, maka kompresi menguat dan kualitas akan menurun. Sedangkan jika QF besar, maka kompresi akan ringan dan kualitas output akan lebih bagus. Berikut perbandingan hasil kompresi gambar di berbagai QF.





5. Ekstraksi Watermark

```
def extract_dct(original_bgr, compressed_bgr, wm_shape, alpha=20):
    """
    Extract watermark from DCT coefficients by comparing original
    and compressed images. Decision rule:
    bit = 1 if coeff_compressed > coeff_original
    bit = 0 otherwise
    """
    def get_Y(bgr):
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb).astype(np.float32)[:, :, 0]

    Y_orig = get_Y(original_bgr)
    Y_comp = get_Y(compressed_bgr)

    H, W = Y_orig.shape
    num_bits = wm_shape[0] * wm_shape[1]
    extracted_bits = []
    bit_idx = 0

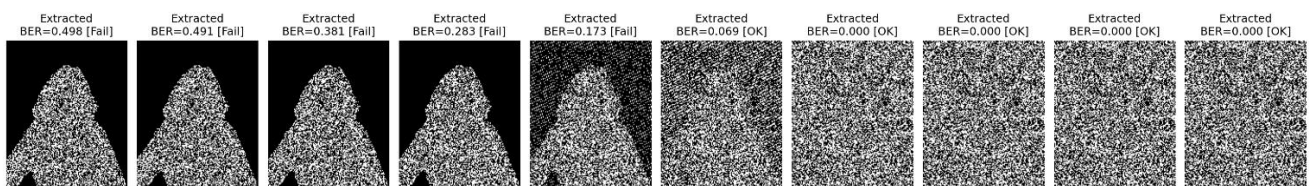
    for row in range(0, H - 7, 8):
        for col in range(0, W - 7, 8):
            if bit_idx >= num_bits:
                break
            dct_orig = cv2.dct(Y_orig[row:row+8, col:col+8])
            dct_comp = cv2.dct(Y_comp[row:row+8, col:col+8])

            pos = MID_FREQ_POSITIONS[bit_idx % len(MID_FREQ_POSITIONS)]
            diff = dct_comp[pos] - dct_orig[pos]
            extracted_bits.append(1.0 if diff > 0 else 0.0)
            bit_idx += 1

    extracted_bits = np.array(extracted_bits, dtype=np.float32)
    if len(extracted_bits) < num_bits:
        extracted_bits = np.pad(extracted_bits, (0, num_bits - len(extracted_bits)))
    return extracted_bits.reshape(wm_shape)

def save_difference_map(original, watermarked, output_dir):
    diff = cv2.absdiff(original, watermarked).astype(np.float32)
    diff_amplified = np.clip(diff * 15, 0, 255).astype(np.uint8)
    diff_gray = cv2.cvtColor(diff_amplified, cv2.COLOR_BGR2GRAY)
    cv2.imwrite(os.path.join(output_dir, "difference_map.png"), diff_gray)
```

Pada bagian ini, *watermark* diekstrak dengan membandingkan koefisien DCT gambar terkompresi terhadap gambar asli sebagai referensi. Berikut hasil ekstraksi watermark untuk rentang QF 10 hingga 100 (interval 10), dan dapat dilihat bahwa pada QF yang rendah (berarti kompresi menguat), watermark tidak dapat diekstraksi secara sempurna.



Evaluasi Kinerja Watermark

Saya mengubah Quality Factor (QF) menjadi beberapa nilai dari rentang QF=10 hingga QF=100 dengan interval 5 dan menghasilkan output berikut.

QF	BER	NC	Extractable?
10	0.4979	0.0041	NO
15	0.4955	0.0091	NO
20	0.4906	0.0188	NO
25	0.4725	0.0550	NO
30	0.3807	0.2385	NO
35	0.3230	0.3541	NO
40	0.2832	0.4335	NO
45	0.2523	0.4954	NO
50	0.1727	0.6547	NO
55	0.1380	0.7241	YES
60	0.0691	0.8618	YES
65	0.0686	0.8628	YES
70	0.0000	1.0000	YES
75	0.0000	1.0000	YES
80	0.0000	1.0000	YES
85	0.0000	1.0000	YES
90	0.0000	1.0000	YES
95	0.0000	1.0000	YES
100	0.0000	1.0000	YES

[RESULT] Watermark CAN be extracted at QF: [55, 60, 65, 70, 75, 80, 85, 90, 95, 100]
[RESULT] Watermark CANNOT be extracted at QF: [10, 15, 20, 25, 30, 35, 40, 45, 50]
[RESULT] => At QF <= 50, JPEG compression destroys the watermark.
[INFO] Robustness plot saved.
[INFO] Visual summary saved.

Dapat dilihat bahwa **watermark dapat diekstrak mulai dari QF 55 hingga 100**. Sedangkan untuk **QF <= 50 tidak dapat diekstrak**. Outputnya, untuk QF >= 55, watermark akan terlihat halus sedangkan jika QF <= 50, gambar akan terlihat buram. Contohnya dapat dilihat pada gambar di bawah ini (gambar saya zoom supaya perbedaan terlihat lebih jelas):

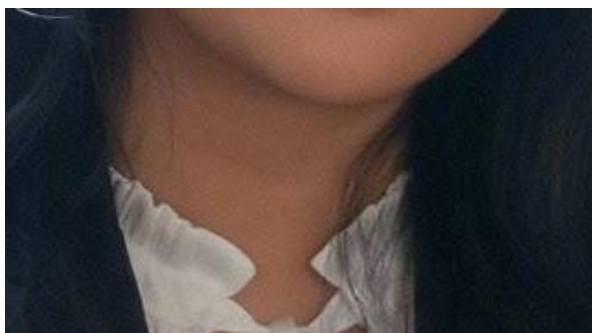
QF 30 (tidak *extractable*)



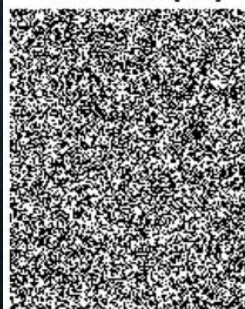
Extracted
BER=0.381 [Fail]



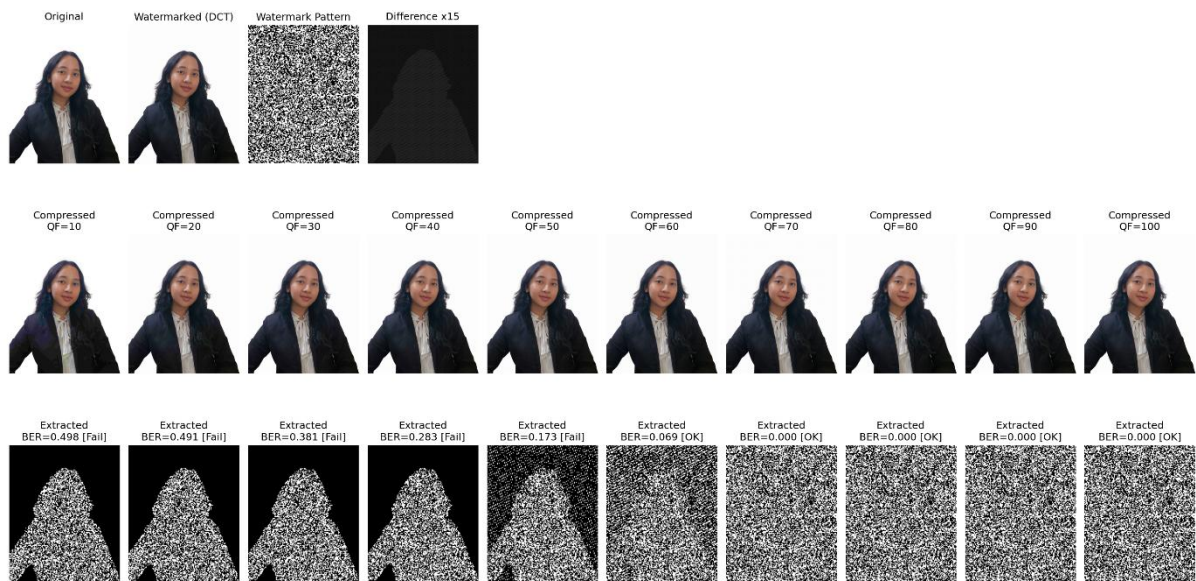
QF 70 (*extractable*)



Extracted
BER=0.000 [OK]



Visual Summary

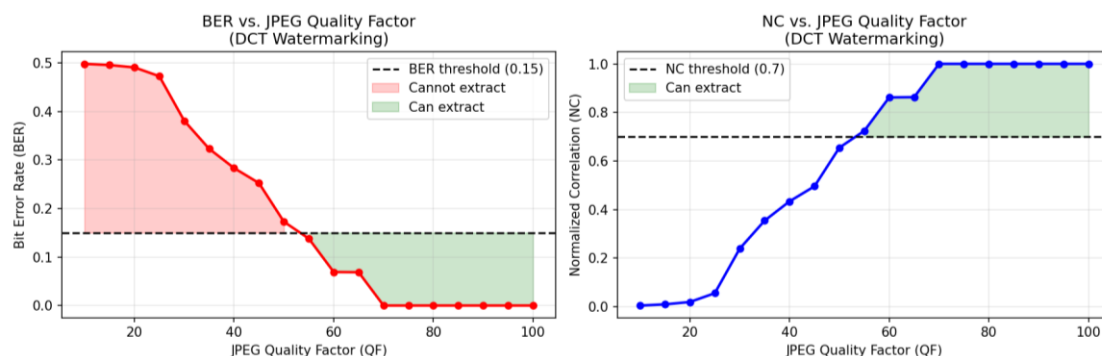


Rangkuman perbandingan kompresi foto dan ekstraksi watermark di berbagai QF:

Pada perbandingan hasil ekstraksi watermark pada berbagai level QF, dapat terlihat bahwa pada **QF ≤ 50** (misalnya pada QF=10 dan QF=30 yang memiliki label *destroyed*), hasil ekstraksi **watermark membentuk siluet** dari foto yang diimport. Hal ini disebabkan karena foto memiliki latar belakang putih polos yang mengandung frekuensi rendah. **Saat kompresi agresif (QF rendah), koefisiennya di-quantize menjadi 0** dan watermark pada area latar belakang hilang sepenuhnya sehingga menyisakan siluet subjek foto.

Di sisi lain, pada QF ≥ 55 (misalnya pada QF = 70 dan QF = 95 yang memiliki label *extractable*), watermark dapat diekstraksi sepenuhnya karena di **QF yang tinggi, nilai table kuantisasinya kecil sehingga hampir tidak ada koefisien yang dibuang**.

Grafik Quality Factor



Pada grafik kiri (BER), Bit Error Rate tinggi di QF yang rendah dan turun drastis setelah QF ≥ 55 (sesuai dengan evaluasi kinerja watermark dan *visual summary* sebelumnya). Sedangkan pada grafik yang kanan (NC), Normalized Correlation naik signifikan setelah QF ≥ 55 yang berarti kesamaan antara watermark asli dengan yang diekstrak berhasil diverifikasi ketika QF lebih dari 55.